ORDER PULSE

Interview questions and sample answers

Interview preparation | Order Pulse | Based on repository code and documented results | 5 October 2026

How to prepare

First: practise the 60-second introduction and the two-headphone order flow without reading.

Next: master questions 11-20. Draw the outbox and inbox transactions and explain two crash scenarios.

Then: practise partition ordering, consumer groups, cross-topic reordering and dead-letter recovery.

Finally: know the endpoints, the documented benchmark limits, and your actual personal contribution.

Demo plan: open localhost:8090, create a small order, observe reserved status and reduced stock. Use a stable Idempotency-Key to demonstrate a retry. This changes local data.

Useful checks: docker compose ps; python3 scripts/smoke.py; mvn -B verify. The smoke test creates data; integration tests require Docker.

Questions to ask the interviewer: How does your team monitor event lag? How do you handle schema changes? Which consistency guarantees matter most for your users?

Sources: README.md; docs/RESULTS.md; OrderController.java; InventoryConsumer.java; AnalyticsConsumer.java. Read these alongside this guide.

Start here: explain the project

1. Tell me about your project in 60 seconds.

Order Pulse is an event-driven order processing demo. A React dashboard lets users order laptops, headphones or keyboards and monitor stock and outcomes. Three Spring Boot services handle orders, inventory and notifications with analytics. Kafka carries events and PostgreSQL stores durable state. The main engineering focus is reliable processing: transactional outboxes, idempotent consumers and stock row locking prevent lost events, duplicate effects and overselling. The dashboard refreshes every two seconds.

2. What business problem does it solve?

It demonstrates how order acceptance, stock reservation and operational reporting can coordinate without requiring every service to be available during the initial HTTP request. Users can see accepted orders move to reserved or rejected outcomes. It is a demo of backend coordination, with no payments, shipping or actual email delivery.

3. What did you personally contribute?

Prepare an honest answer: I personally worked on [components you actually implemented], made [specific decision], and verified it through [checks you actually ran]. Explain one file and one failure scenario in depth. Do not claim that you independently wrote or deployed every component if you used assistance or did not perform those steps.

4. Walk through a two-headphone order.

The dashboard posts the SKU and quantity with an Idempotency-Key. Orders commits the order and OrderCreated outbox record together. The publisher sends the event to Kafka. Inventory locks the headphone stock row, deducts two units if available, and commits a reservation plus completion outbox record. Notifications consumes the events and updates SQL projections and a durable notification record. The next dashboard poll displays the result.

5. Why use three services?

Orders owns request acceptance, Inventory owns stock decisions, and Notifications owns notification records and analytics. Separate ownership makes responsibilities clear and allows independent evolution. For a small application, a modular monolith could be simpler; this project uses services to demonstrate distributed reliability patterns.

Architecture and data ownership

6. Why Kafka instead of only REST calls?

Kafka decouples event producers from consumers and retains events for later processing. Inventory can process accepted orders after recovering from downtime. REST is still used at the browser boundary. The tradeoffs are eventual consistency, more infrastructure and harder debugging.

7. What does each database contain?

One PostgreSQL server hosts three separate databases. Orders stores orders and its outgoing events. Inventory stores stock, reservations, inbox markers and outgoing events. Notifications stores deduplication markers, notification records and analytics projections. Services do not query another service’s database.

8. Is the dashboard immediately consistent?

No. Order acceptance and inventory completion happen in separate transactions and services. The order can appear as processing before its completion event reaches analytics. The dashboard polls every two seconds, adding display delay. GET /api/stock reads authoritative inventory; analytics also maintains event-driven stock snapshots.

9. Is this CQRS or event sourcing?

It has CQRS-style read projections: analytics tables are shaped for dashboard queries while business services own writes. It is not a full event-sourced system because business state is stored directly in relational tables rather than reconstructed solely from an event log.

10. What technologies are used and why?

Spring Boot provides HTTP endpoints, validation, transactions and Kafka integration. JdbcTemplate makes SQL and locking explicit. PostgreSQL provides durable relational data and uniqueness constraints. Kafka transports events. React renders the dashboard, Nginx proxies same-origin API requests, and Docker Compose runs the local stack.

Reliability: the most important questions

11. What problem does the transactional outbox solve?

Saving an order in SQL and independently publishing to Kafka creates a dual-write gap: one operation may succeed while the other fails. The outbox stores the order and event in the same SQL transaction. A separate publisher sends pending events. This guarantees a committed order has a durable publication intent, but a crash can still cause an event to be sent twice.

12. What if the publisher crashes after Kafka accepts the event?

If SQL has not yet recorded that the event was published, the publisher will send it again after restart. This is why consumers must tolerate redelivery. Producer idempotence helps Kafka producer retries, but it does not make Kafka and SQL one atomic transaction.

13. How do you prevent duplicate HTTP orders?

The client sends an Idempotency-Key. A database uniqueness constraint permits only one order per key. Retrying the same key with the same SKU and quantity returns the existing order. Reusing the key with a different payload returns HTTP 409. A new key represents a new request even if its payload is identical.

14. How do consumers prevent duplicate effects?

A unique inbox marker identifies a consumer and event ID. That marker, stock changes or projection changes, and any outgoing event commit in the same transaction. A redelivered event with a committed marker is ignored. Additional order-ID constraints protect business effects such as reservations and notification records.

15. Why must the inbox and business update share a transaction?

If the inbox commits first and the business update fails, a retry could incorrectly skip unfinished work. If the business update commits first and the inbox fails, a retry could apply the effect twice. Committing both together means success records both, and failure rolls back both.

16. Do you guarantee exactly-once processing?

I describe this as at-least-once delivery with idempotent SQL effects. Events can be delivered more than once, but successful duplicate deliveries do not repeat the protected business updates. I do not claim globally exactly-once delivery across Kafka and PostgreSQL.

17. How do you prevent overselling?

Inventory selects the stock row using SELECT ... FOR UPDATE inside the reservation transaction. Competing transactions for that SKU wait for the lock, then check the latest available stock. The check, deduction, reservation and outgoing event commit together. Unique order reservations also prevent repeating a completed reservation.

18. What happens when stock is insufficient?

Inventory records a rejected reservation and produces StockRejected without deducting stock. This is an expected business outcome. It does not need retries or a dead-letter topic. The dashboard eventually shows the rejected status.

19. What happens if Kafka or Inventory is unavailable?

If Kafka is unavailable, committed outgoing events remain pending in SQL for the publisher to retry. If Inventory is unavailable, Kafka retains events for processing after it returns, subject to retention. Orders can remain pending during recovery. Monitoring pending outbox age and consumer lag would be important in production.

20. What if the consumer commits SQL but crashes before its offset is committed?

Kafka may deliver the record again. The committed inbox marker lets the consumer recognize that the SQL effects already succeeded and avoid applying them twice. If SQL fails before commit, the marker and effects roll back and the retry can safely attempt the work again.

Kafka and event ordering

21. What events and topics are used?

The main types are OrderCreated, StockReserved and StockRejected. Order events use orders.events; completion events use inventory.events. Envelopes contain an event ID, type, order ID, SKU, quantity, optional stock and UTC occurrence time. Failed records can be sent to the corresponding .DLT topic.

22. Why use SKU as the Kafka key?

A SKU key routes events for the same product to the same partition within a topic. Kafka preserves partition order; it does not guarantee a global order across partitions or across topics. SQL row locking still protects stock updates.

23. What is a consumer group?

Consumers with the same group ID share a topic’s partitions. Different groups each receive their own view of the events. Inventory and Notifications use separate groups so both can receive order events. With three partitions, at most three consumers in a group can actively read that topic at once.

24. What if StockReserved arrives before OrderCreated?

This can happen because the two events are on different topics. Analytics can create a projection from the completion event. A later OrderCreated fills in creation time without resetting the final status to pending. Order counting also deduplicates by order ID, even if a creation event is repeated under a new event ID.

25. What are retries and dead-letter topics for?

Unhandled processing failures get up to three retries at 500 ms intervals, then go to the source topic’s .DLT with diagnostic headers. Examples include malformed envelopes or unknown SKUs. A failed DLT publication must not be treated as successful recovery. Fix the cause before replaying valid records with their original identity.

26. How are stale stock snapshots handled?

The analytics stock projection uses the smaller of its existing and incoming stock values. This works because the current model only deducts stock. If restocking is introduced, this rule would be wrong; I would use a version or sequence per SKU to reject older updates while allowing legitimate increases.

API, dashboard and testing

27. Which endpoints should you know?

POST /api/orders creates or retrieves an idempotent order. GET /api/orders/{id} returns the accepted order record, not its final reservation projection. GET /api/stock returns available inventory. GET /api/metrics provides dashboard aggregates and recent-order statuses. Nginx exposes the dashboard and APIs through port 8090.

28. How do you validate an order?

The API allows LAPTOP, HEADPHONES or KEYBOARD and a quantity from 1 to 1,000. It requires a nonblank Idempotency-Key of at most 128 characters. New valid requests return 201; identical retries return 200; conflicting payloads for an existing key return 409. An unknown order ID returns 404.

29. How does the dashboard update?

React fetches /api/metrics and /api/stock every two seconds. It shows order totals, orders per minute, reserved and rejected counts, available inventory, a recent activity chart and recent orders. Polling is simple for this demo; server-sent events or WebSockets could reduce display delay for a larger live application.

30. How was reliability tested?

The repository documents nine passing integration tests using real Kafka and PostgreSQL through Testcontainers. Coverage includes duplicate delivery, HTTP retry conflicts, insufficient stock, rollback and DLT delivery, cross-topic reordering, duplicate aggregation and stale stock replay. A separate HTTP smoke test checks one order, one stock deduction and one final projection after an HTTP retry. These are documented results, not checks rerun while preparing this guide.

31. What performance result can you defend?

The documented local benchmark completed 1,000 unique orders in 5.61 seconds, about 178.24 completed orders per second. It sent 2,000 HTTP requests with repeated idempotency keys using 20 workers and verified zero duplicate order effects. It includes the pipeline through final SQL aggregation. This is one local run, not a sustained capacity limit, raw Kafka throughput or an AWS result.

32. What do Docker Compose and CI provide?

Compose runs the dashboard, three application services, Kafka and PostgreSQL. Kafka and PostgreSQL health checks gate service startup; named volumes preserve data. The GitHub Actions workflow is configured for Java integration tests, React builds and an HTTP smoke test. A configured workflow is not proof that GitHub-hosted CI has run.

Tradeoffs, follow-ups and production readiness

33. How would you scale it?

Add partitions and consumers where throughput requires it, and measure lag and database contention. SKU partitioning and row locking mean a very popular product can remain a bottleneck. The current design assumes one outbox publisher per service; multiple publishers need coordinated claiming and careful ordering. Production would also need multiple brokers and resilient database hosting.

34. What would you improve before production?

Add authentication and authorization, TLS and secret management, monitoring of lag and outbox age, alerting and tracing, backups, retention policies and high availability. Actual email delivery needs a provider integration with safe retries. Restocking needs ordered stock versions. I would prioritize improvements based on business risk and measured load.

35. Have you deployed this on AWS?

The repository contains Terraform and deployment instructions, and documents successful Terraform validation. It explicitly says no AWS resources were created and the live demo URL is pending. I would describe the deployment plan honestly rather than claim a running cloud deployment.

36. How would you debug an order stuck as pending?

Start with the order ID and verify the accepted order. Check whether its outbox record was published, then inspect producer errors, Inventory consumer lag and any DLT record. Check the reservation and completion outbox, followed by Notifications processing and its projection. Finally check dashboard API responses. Follow one order through the system rather than restarting every service blindly.

37. What is the hardest design challenge?

A strong example is keeping SQL effects correct despite retries and cross-topic reordering. The solution combines atomic outbox writes, inbox deduplication and projection updates that preserve a final status when creation arrives late. Explain the failure before the fix, then the invariant and the integration test that demonstrates it.

38. Why not choose a simpler architecture?

For a small business, a monolith with one database can be a better starting point. This architecture is useful when asynchronous processing and separate service ownership justify the operational cost. I can explain its reliability patterns while acknowledging the extra infrastructure and eventual consistency.